AZIMI INNOVATION LAB

AI ARCHITECT | AMIN AZIMI

END-TO-END SYSTEM DEVELOPMENT

Graph-Native Knowledge Engineering & Evidence-Grounded RAG Platform

Architecture Baseline — Requirements 1.0

Architecture specification — a design artifact, not an implemented product

Amin Azimi | AI Architect | Azimi Innovation Lab

November 2026

© 2026 Amin Azimi. All rights reserved.

Requirements Baseline 1.0 is approved. This document is not proof of implementation, and no executable test has been run against it.

0 Reading Guide and Document Status

This document is built on approved design decisions. Each clause carries one of the labels below, so that the line between “approved”, “proposed” and “proven” is always clear.

REQ owner-approved requirement ADR approved architecture decision DESIGN proposed contract ASSUMPTION untested assumption REF external reference EVIDENCE real execution evidence

Document control

Item	Value
Requirements baseline	1.0 — owner-approved; reconstructed and not presented as a verbatim quotation of a historical version
Author and rights holder	Amin Azimi — AI Architect — Azimi Innovation Lab
Classification	Public portfolio document; all rights reserved (see Section 18)
Date	November 2026
Execution status	No implementation, security validation, benchmark, recovery exercise or legal review has been performed.

Contents

0 Reading Guide and Document Status 2
1 Vision and Scope 4
2 Trust and Responsibility Boundaries 5
3 High-Level Architecture and Decisions 7
4 Knowledge Model and Ontology 9
5 Repositories, Spaces and Multi-Tenancy 13
6 Ingestion, Processing and Publication 14
7 Hybrid Retrieval and GraphRAG 16
8 Security and Operational Governance 21
9 Persian-Language Processing 25
10 Deletion, Retention and Immutability 28
11 Evaluation and Execution Evidence 30
12 Disaster Recovery and Proposed Targets 32
13 Dual-Mode Delivery Roadmap 32
14 Formal 32-Requirement Matrix and Acceptance Criteria 34
15 Technical Scenarios and Negative Tests 42
16 Audit Controls 44
17 Technical References and Citation Status 45
18 Rights Notice 47
19 Related Designs by the Same Author 48

1 Vision and Scope

REQ DESIGN

This platform turns versioned source documents into structured knowledge, an evidence graph and search indexes. The source document and its immutable revision are the “source of truth”; the graph, full-text search and vector search are rebuildable projections of it. Answers must be built only from authorized evidence that is consistent with a single Publication; when evidence is insufficient, the system must refuse to give a definitive answer. This project is an architecture specification and makes no claim of production readiness.

In scope

- Document ingestion; immutable source revisions; validated metadata; controlled ontology.
- Claims and relations with provenance; atomic publication manifests.
- Hybrid retrieval; evidence-grounded answers; Tenant / Space authorization.

- Audit; evaluation; disaster recovery; implementation and release gates.
- Persian-language processing, deletion and retention policy, and the claim-support validation method (Sections 7, 9 and 10).

Out of scope (without separate owner approval)

- Adopting Neo4j; active multi-region deployment; Kubernetes-first deployment.
- Unbounded autonomous execution; publication of confidential knowledge.
- Any commercialization or implementation right arising from this PDF itself.

2 Trust and Responsibility Boundaries

ADR DESIGN

User identity, Tenant and permissions are derived only from server-side validation, never from client claims or document frontmatter. Document text, frontmatter, retrieved data, worker messages and language-model output are all potentially untrusted input and are treated as data, not instructions.

Roles (eight roles)

Owner, Tenant Administrator, Knowledge-Space Steward, Contributor, Reader, Reviewer, Platform Operator and Worker Service Identity. No role bypasses explicit resource authorization.

Trust boundaries

- External repository connectors; uploaded documents; extracted text; metadata and frontmatter.
- Retrieval results; language-model input and output; cross-space Bindings.
- Internal worker messages; object storage; backup media; administrator controls.

Foundational invariants

- Once a Revision is recorded, the source is immutable.
- Every published derived object references a Revision and its provenance.
- Every read executes within an authorized Tenant / Space / Publication scope.

- Permission revocation is independent of Publication activation and takes effect immediately.
- Retries are idempotent; a partial ingestion never becomes a visible Publication.
- Default principle: deny — for an unknown scope, stale authorization, an unsupported citation, or an inconsistent snapshot.

3 High-Level Architecture and Decisions

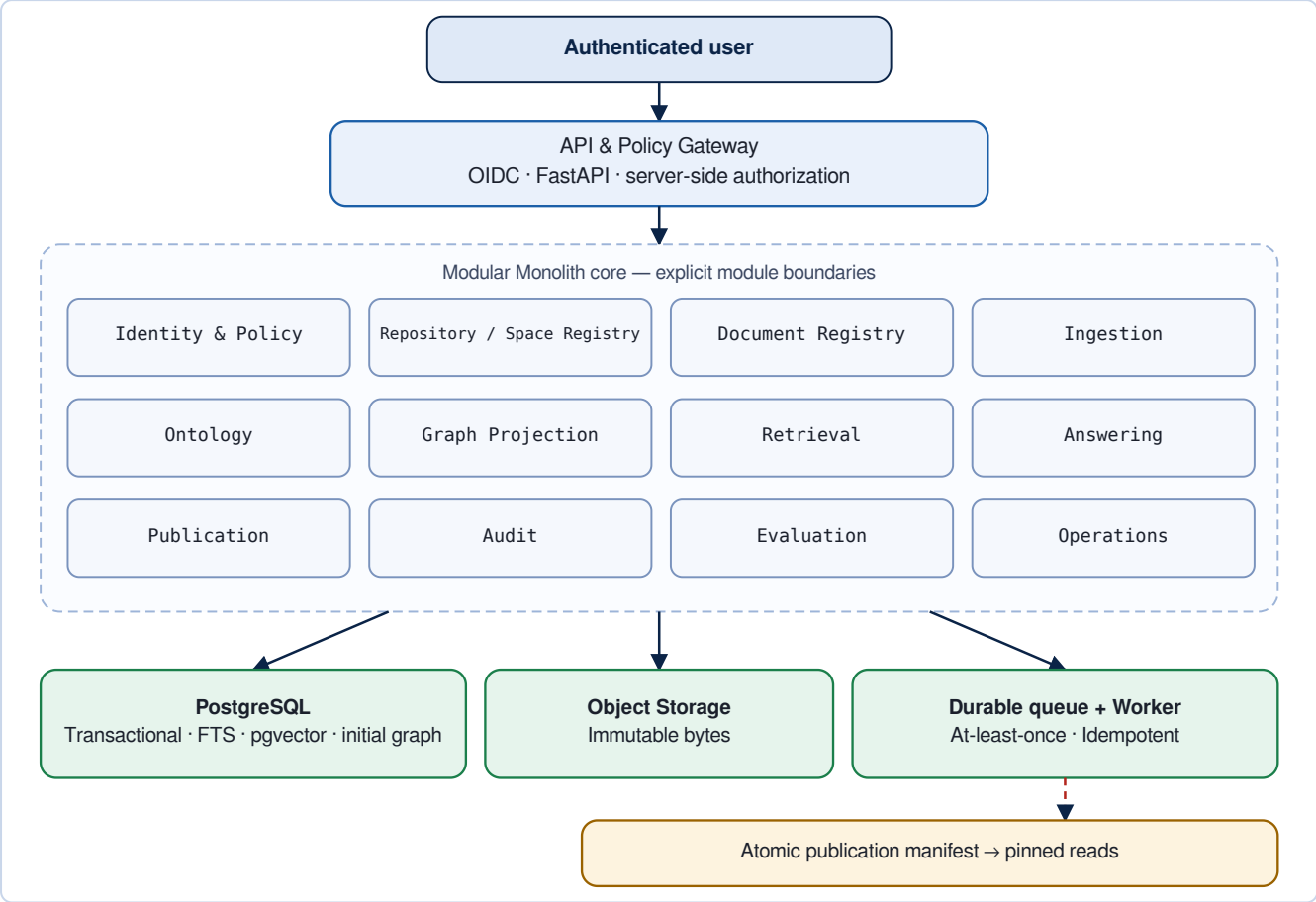
ADR DESIGN

Decision	Summary
ADR-01	Modular Monolith with asynchronous workers and a durable queue. Modules: Identity & Policy, Repository/Space Registry, Document Registry, Ingestion, Ontology, Graph Projection, Retrieval, Answering, Publication, Audit, Evaluation, Operations. At-least-once delivery with an idempotency key; a retry must not create a duplicate Revision.
ADR-02	PostgreSQL-first persistence: transactional metadata, full-text search, pgvector, and graph nodes/edges in PostgreSQL. Object storage holds immutable bytes and large artifacts. Neo4j is a future option subject to benchmarking, not an approved dependency.

Decision	Summary
ADR-03	FastAPI, a React/TypeScript client, OpenAPI contracts, OIDC authentication, OpenTelemetry observability, and portable containerized deployment. Versions, licenses and dependency risks must be recorded in the implementation manifest.
ADR-04	Single region with backup in an independent failure domain and Point-in-Time Recovery. Proposed targets (unmeasured): primary-data RPO of at most 15 minutes, RTO of at most 4 hours, projection rebuild within 24 hours.

Logical data flow: authenticated user → API and Policy Gateway → authorized Tenant/Space scope → Registry and immutable Revision storage → ingestion and validation workers → ontology/provenance graph and text and vector indexes → verification of the publication candidate → atomic Manifest activation → bounded hybrid retrieval → policy route → evidence verification → answer or abstention → audit and metrics.

Figure 1 — Architecture and module boundaries



Arrow directions are explicit; the layout reads top to bottom. The diagram shows the approved architectural relationships.

4 Knowledge Model and Ontology

DESIGN

Entities

Entity	Definition
Document	A stable identifier independent of path and file name; Tenant ownership and its link to a Repository.

Entity	Definition
DocumentRevision	Content hash, byte length, content type, creation and ingestion times, source provenance, validation status; immutable.
Section	An ordered content range anchored to one Revision.
Concept	A controlled term together with its aliases.
Claim	A normalized proposition with provenance and a validation status.
Evidence	A typed link to a Revision, a range, and the extraction process.
Source	A reference to the origin and the source's permissions.
Topic	A controlled classification.
ArchitecturePattern	A described, reusable pattern; never presumed to be a reference requirement.

Relations (eleven relations)

REFERENCES, CONTAINS, MENTIONS, SUPPORTS, CONTRADICTS, RELATED_TO, DEPENDS_ON, IMPLEMENTS, DERIVED_FROM, SUPERSEDES, HAS_REVISION

SUPPORTS and CONTRADICTS are not valid without inspectable evidence and are never inferred from similarity alone. A valid extracted relation is not automatically a “confirmed fact”.

Evidence span contract

- Offsets are UTF-8 bytes in one immutable, canonical Revision; a range is written `[start, end_exclusive)`.
- Condition: `0 ≤ start < end ≤ byte_length`, the exact Revision hash, and the ability to re-extract the quoted bytes.
- Unicode normalization or parser transformations must never silently shift canonical offsets; a textual rendering is a derived view, not the primary provenance.

Canonical bytes and hashing

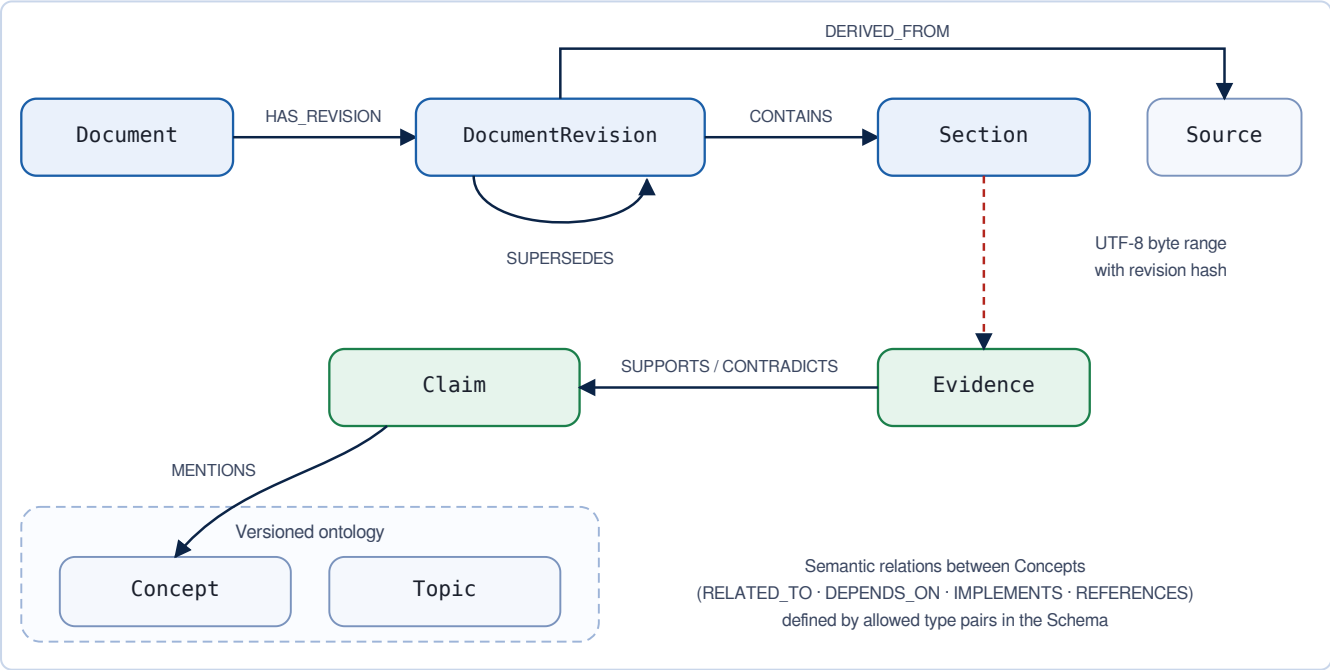
The canonical bytes of a Revision are the bytes as received, after UTF-8 validity checking, with no Unicode normalization, no line-ending conversion and no BOM removal. Two hashes are kept distinct: the Revision digest covers the whole canonical byte sequence of one Revision, and an optional span digest covers only the bytes of one evidence range. Validation runs in that order: Revision digest first, then range bounds, then the span digest when one is stored. Baseline 1.0 accepts Markdown documents encoded as UTF-8 only; PDF, HTML and other binary formats are outside this baseline and would need a separate extraction contract. The proposed hash function is

SHA-256, stored in hexadecimal with an algorithm prefix so that the algorithm can be migrated. The final algorithm and format must be recorded in the implementation Manifest.

Controlled ontology

A stable core plus a versioned registry of domain extensions. Every proposed extension must declare its namespace, types, predicates, compatibility, migration and reviewer, and only an approved version becomes active. An ambiguous entity merge is reversible and goes to review, never a silent merge.

Figure 2 — Knowledge, relations and evidence provenance



Edge names are the formal relations of Section 4. A dashed line is a connection property (not a relation). Each arrow points from the source entity to the target. The exact type pairs for relations between Concepts are fixed during schema design.

5 Repositories, Spaces and Multi-Tenancy

REQ DESIGN

The Tenant is the isolation boundary. A Repository is a collection of external or managed sources. A Knowledge Space is the authorized scope for publication and retrieval. A Repository Binding is an explicit many-to-many relationship between repositories and spaces, and each Binding has its own policy, source-version selection and provenance. One repository can feed several spaces without any space gaining rights over another; one space can also combine several repositories and preserve source-level rights.

A policy check ties together the acting principal, Tenant, Space, Binding, operation, Revision, Publication and the current revocation epoch. A policy decision must be recomputed or invalidated when permissions change. A cache key must encode the permission-relevant scope and version; a globally shared result for Tenant-private evidence is forbidden.

Negative cross-tenant and cross-space tests must cover

- Document and result counts; autocomplete.

- Backlinks; graph neighbors.
- Search snippets; citations.
- Error shape and timing behavior; cache.

6 Ingestion, Processing and Publication

DESIGN REQ

Proposed state machine (Figure 3):

```
DISCOVERED → FETCHED → IMMUTABLE_REVISION_STORED →
VALIDATED → EXTRACTED → PROJECTED → VERIFIED →
CANDIDATE_READY → PUBLISHED
```

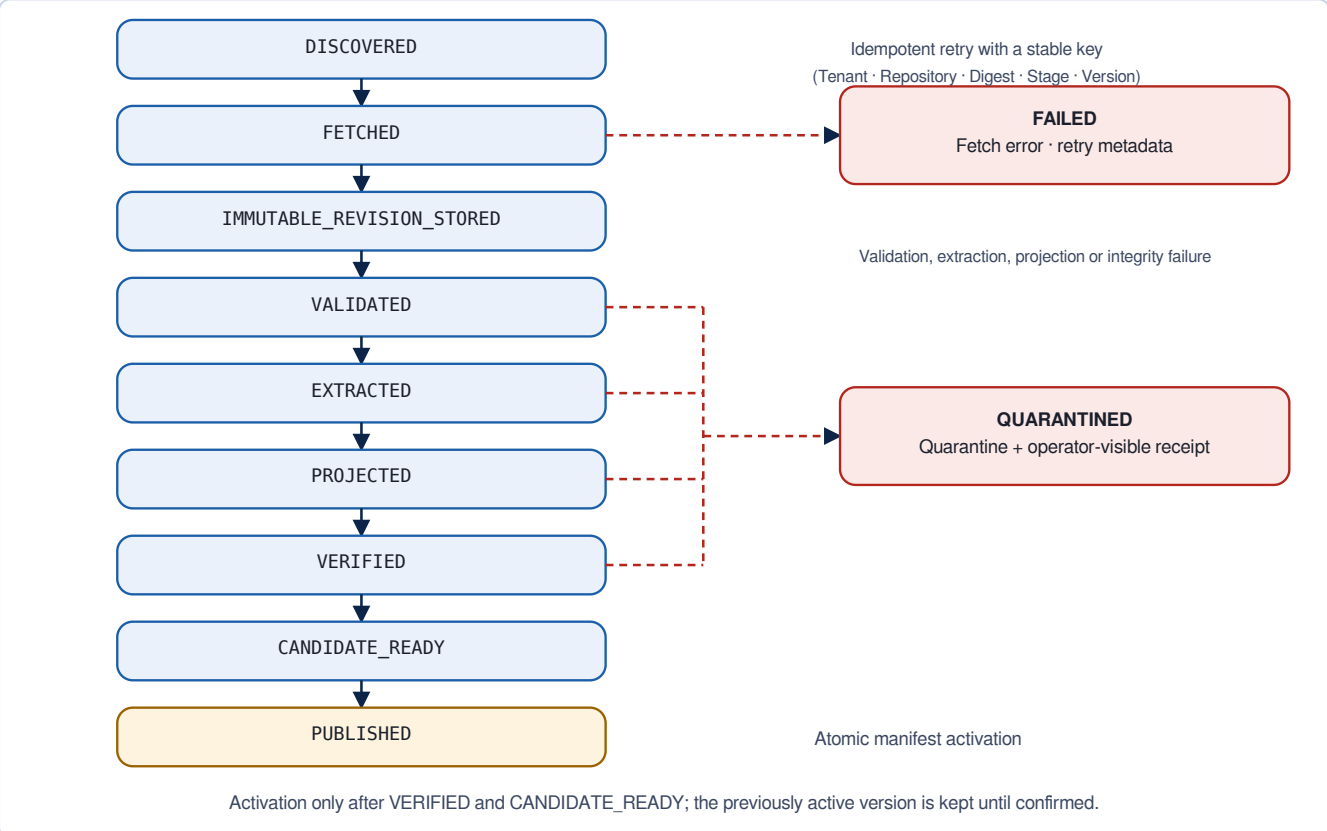
QUARANTINED and FAILED paths, with retry metadata, are mandatory. Invalid frontmatter is untrusted and cannot grant access. Links of the form `[[document-id]]` are interpreted against stable identifiers; an unresolved link remains an explicit diagnostic. Backlinks are derived from the current authorized links and are not manually maintained truth.

The publication Manifest contains: Tenant, Space, Manifest identifier, the set of Revisions, the ontology version, build identifiers of the graph/text/vector indexes, embedding model, version and dimensions, extraction model and version, the validation report, and immutable content hashes. The Manifest

pointer is activated transactionally only after all mandatory checks pass. Every read request pins one Manifest; a missing or stale projection means a controlled failure or a safe degraded path, never a mixture of generations. Permission revocation is immediate even with a pinned Manifest.

Idempotency rests on the key (Tenant, Repository, source Revision hash, stage, model/schema version). A retry under at-least-once delivery must have no side effects; poisoned jobs are quarantined with an operator-visible receipt.

Figure 3 — Atomic ingestion and publication



The main path runs top to bottom; dashed red arrows are failure paths.

7 Hybrid Retrieval and GraphRAG

DESIGN REQ

Query path: authentication → resolve authorized scope → select the pinned Manifest → normalize the query → run PostgreSQL FTS, vector retrieval and bounded graph traversal → per-source authorization filtering → Rank Fusion → evidence reranking → provenance and citation validation → route decision. Every candidate source must belong to one Publication and pass a live access check.

Route	Condition and behavior
Route A	A deterministic structured answer, when the query can be answered directly from validated structured records.
Route B	Evidence-grounded language synthesis with strict citations and post-generation validation that each claim is supported.
Route C	Abstention with an explanation, when evidence is missing, conflicting, unauthorized or unverifiable.

The language model must not execute instructions embedded in retrieved text. A confidence score never overrides a hard policy failure.

Citation contract

Every citation contains the stable document identifier, the identifier and hash of the immutable Revision, the canonical byte range, the publication Manifest, and an authorized display label. Every material claim in an answer must be supported by available evidence. An unsupported answer is rejected, corrected or converted to an abstention, never silently published.

Claim-support validation method

Claim validation runs in three layers, and each layer can reject the answer:

- Layer 1 — rule-based: every citation must resolve to a valid byte range and hash; the bytes are re-extracted and live authorization is checked. This layer is deterministic and does not depend on a model.
- Layer 2 — textual entailment (NLI) or a language-model judge: runs only on the re-extracted bytes and a single claim, never on the whole retrieved text.
- Layer 3 — injection resistance: input and output controls around the Layer 2 judge, not a second model. The judge input is only the claim and the evidence range; the judge

output must come from a closed schema (supported / unsupported / contradicted), and anything outside the schema is rejected. The judge never has tools or executable permissions, and an instruction embedded in the evidence must not change its behavior.

No specific model or package is pinned for Layers 2 and 3 in this document; that choice is deferred to the P5 benchmark, and the final name and version are recorded in the Manifest. The interface is fixed: the input is a single claim and the re-extracted evidence bytes; the output is one of supported / unsupported / contradicted. Baseline selection criteria: accuracy on supported and contradicted samples in Persian and English, resistance to prompt injection, latency and cost, the possibility of on-premises operation, and license terms. Two candidate classes are evaluated: an open-license multilingual NLI model, and a language model acting as a judge with a closed schema.

The judge acceptance threshold must be set on a representative evaluation set for this platform and approved by the owner before release; no numeric value is given in this document, deliberately.

Evidence-conflict policy

The approved default is Route C. As a configurable option per Space, a “two-sided presentation” can be enabled: both conflicting views are presented with full citations and a conflict label. Enabling this option requires approval by the Space owner and a record in the Manifest.

Vector retrieval and authorization

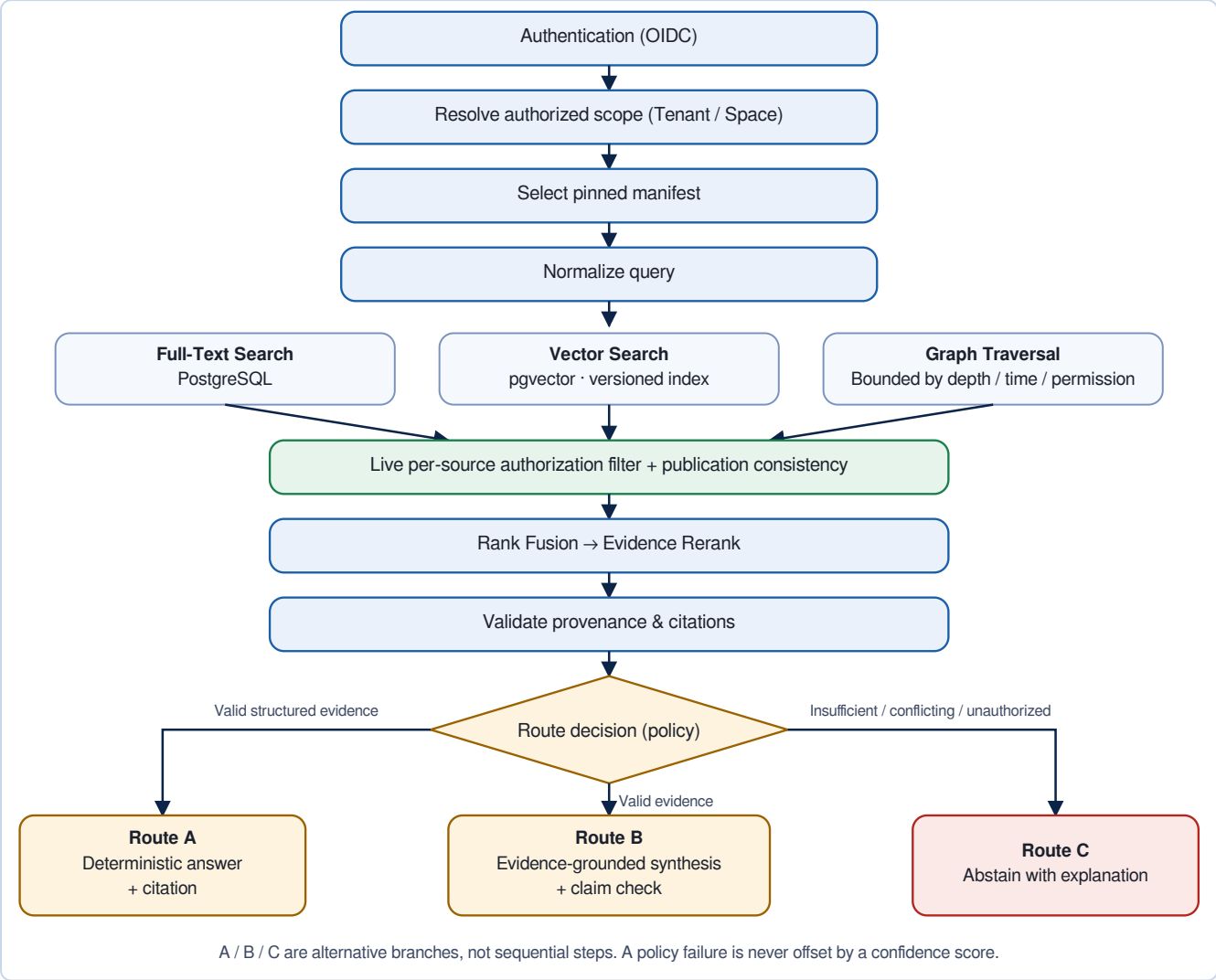
- Post-filtering by permission can reduce recall; therefore the Tenant and Space filter must be applied before or during vector search (a partition or partial index per Tenant/Space).
- The index type (HNSW or IVFFlat), its parameters, and the effect of filtering on recall must be benchmarked and recorded in P4; until then the choice is not final.
- Every embedding is labeled with its model, version and dimensions, and incompatible indexes are never mixed.

Graph traversal bounds (proposed assumption)

Initial proposed configuration values (ASSUMPTION, before benchmarking): maximum depth 3, a cap of 5000 visited nodes per query, and a 300 millisecond timeout. These are configuration defaults, not acceptance thresholds; exceeding any

bound must cut the traversal off and record an event. These values must be per-Space configuration parameters that can be changed without redeploying code, not hard-coded constants; a truncated result is returned labeled “incomplete due to bound”, and the final value is recorded in the Manifest after the P4 benchmark. A result truncated by a bound is never presented as complete: the answer is either an abstention, or it is issued with an explicit incompleteness warning and only when the remaining evidence is sufficient under the route policy.

Figure 4 — Hybrid retrieval and answer decision



A / B / C are alternative branches, not sequential steps. A policy failure is never offset by a confidence score.

Branches A, B and C are alternatives to one another and exactly one of them is chosen.

8 Security and Operational Governance

ADR DESIGN

- Identity: OIDC validation, short-lived tokens, Tenant derived by the server.

- Authorization: deny-by-default in the API, workers, database, graph, text and vector search, object retrieval and response serialization.
- Database defense: PostgreSQL Row-Level Security with a tested role/Tenant reset in the connection pool, least-privilege roles and a separated service account. RLS is not a substitute for application policy.
- Controls: encryption in transit and at rest; secret management and rotation; upload and parser limits; dependency and container scanning; SBOM; append-only audit as far as possible; log scrubbing; an incident runbook; an immutable backup policy; tested Restore and PITR; rate and cost limits; human review of ontology changes and ambiguous merges.
- Observability: a request Trace ID links authorization, ingestion jobs, publication, retrieval route, citation validation and the answer receipt without exposing confidential text.

Every control requires a threat-based negative test, not merely a claim.

Data flow to an external language model

Whenever a language model or embedding is served by an external provider, each Space must specify the following before activation: an allow-list of providers, the processing region, a no-retention and a no-training-on-data commitment where the provider offers one (these are separate commitments that differ by provider, and none is assumed), and a data-processing agreement. Sending text to a provider that is not on the Space's allow-list is forbidden. Only authorized, minimal fragments reach the model; the legal analysis of this section remains in `AUD-09`.

Formal threat model — STRIDE summary

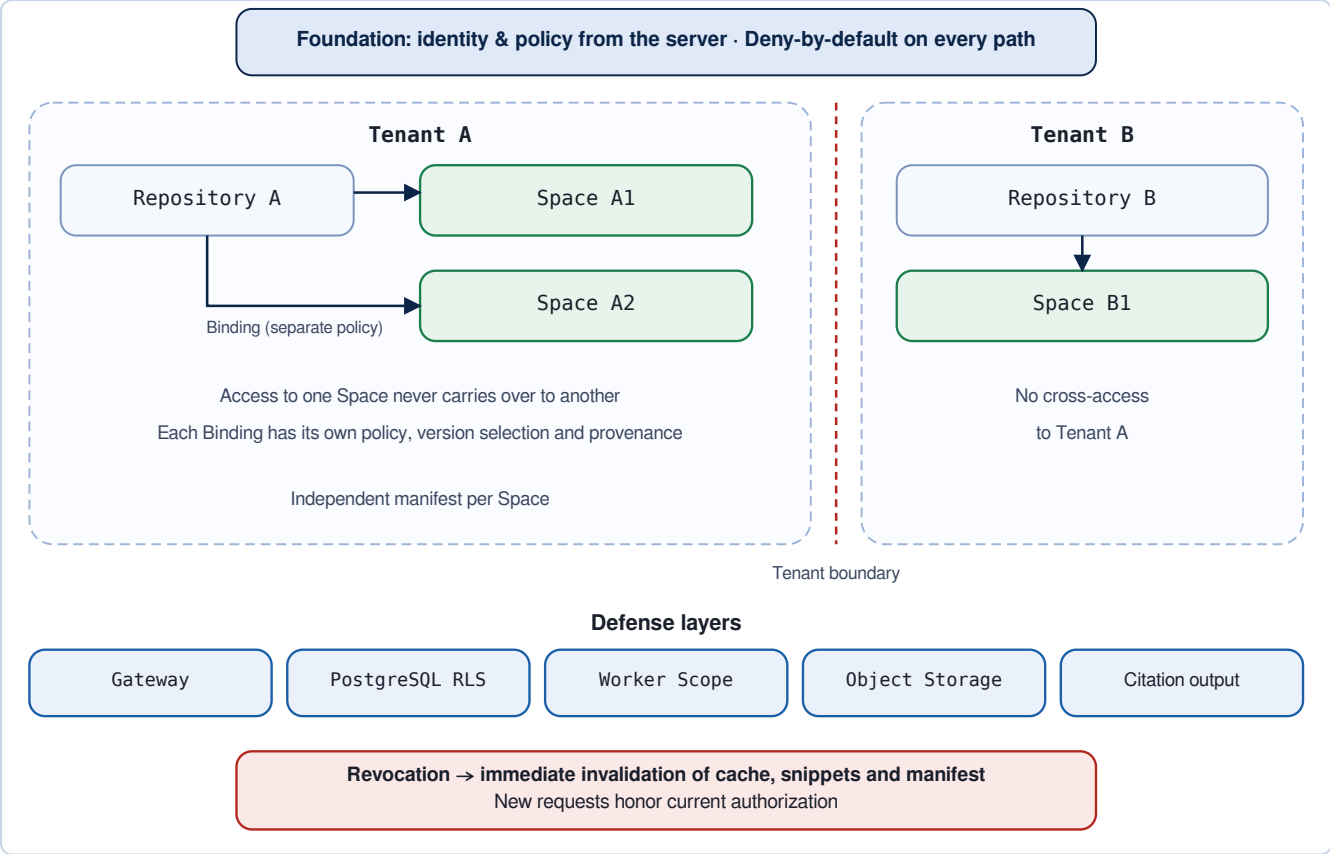
Asset	Threat	Type	Control and test
Identity and Tenant	Spoofing a Tenant via frontmatter or a client claim	Spoofing	Tenant only from server validation; test `MT-06`
Source Revision	Silent change of content or range	Tampering	Immutability, hashing, range validation; `FT-02`, `FT-11`
Sensitive actions	Denying a change or an access	Repudiation	Append-only audit; `MT-12`
Private evidence	Leakage through cache, counts, snippets, citations	Information Disclosure	Permission-aware cache keys; `MT-09`

Asset	Threat	Type	Control and test
Worker and queue	Message poisoning or heavy load	Denial of Service	Quarantine, rate and cost caps, idempotency; FT-13
Tenant and Space	Privilege escalation through a Binding	Elevation of Privilege	Binding with separate policy; MT-05
Language model	Prompt injection in retrieved text	Tampering / EoP	Text treated as data, closed schema; FT-18

Assumed adversaries: a low-privilege user inside a Tenant, a user of another Tenant, a malicious document author, an over-privileged operator, and an external provider. This table is a starting point and must be completed and versioned from P1 through P6 with the real assets and boundaries.

Privacy: the inventory of data, retention, deletion and export, and the processing roles require a project-specific legal analysis. No claim of GDPR or AI Act certification is made.

Figure 5 — Tenant and Space security boundaries



Revocation of access operates independently of the Manifest and has an immediate effect on the cache, snippets and subsequent answers.

9 Persian-Language Processing

ASSUMPTION

Because this document and an important part of the target knowledge are Persian, retrieval recall depends directly on this section. The items below are proposed designs and must be confirmed against a Persian evaluation set before P4.

- Normalization for indexing, separate from the canonical bytes: unify Arabic and Persian Yeh (U+064A to U+06CC) and Kaf (U+0643 to U+06A9), map Persian and Arabic-Indic digits to ASCII, harmonize the zero-width non-joiner (ZWNJ, U+200C), and remove Tatweel and optional diacritics.
- The normalized index is only a derived view; offsets still refer to the canonical bytes.
- PostgreSQL FTS has no built-in Persian dictionary or stemmer. Proposal: a simple configuration with custom normalization, or a tokenizer extension with ICU support. The final choice is made in P4 with a benchmark.
- The embedding model must be multilingual with documented Persian support; its name and version are recorded in the Manifest.
- The evaluation set must include Persian queries and documents, mixed Persian/English text, and phrases containing ZWNJ.

Proposed pattern: normalized search with canonical bytes preserved

The pseudocode below (illustrative design only, not ready-to-run code) shows how FTS works on the derived, normalized text while evidence is always recorded against the canonical bytes. No offset from the normalized text is stored or used as provenance; the mapping table exists only to translate a result back to canonical bytes. PostgreSQL FTS is not assumed to return exact character spans: locating a hit inside the normalized text is a separate alignment step (a DESIGN item to be settled in `P4`), and `hit.start_char` and `hit.end_char` stand for the output of that step.

```
# Canonical bytes are never modified or re-encoded.
canon = revision.bytes                          # immutable UTF-8

def normalize_with_map(chunk_text, chunk_byte_start):
    out, omap = [], []          # omap[i] = canonical byte offset of out[i]
    for ch, byte_off in iter_chars(chunk_text, chunk_byte_start):
        for n in fold(ch):      # unify Arabic/Persian yeh and kaf;
            out.append(n)       # digits->ASCII; drop Tatweel/harakat;
                                # ZWNJ per policy
            omap.append(byte_off)
    return "".join(out), omap

# Index time (derived data only)
norm, omap = normalize_with_map(chunk.text, chunk.byte_start)
store(chunk_id, tsv=to_tsvector("simple", norm),
      omap=compress(omap), byte_start=chunk.byte_start,
      byte_end=chunk.byte_end, manifest=manifest_id)

# Query time
hit = fts_match(normalize(query))      # match on derived text only
i, j = hit.start_char, hit.end_char    # positions inside `norm`
start = omap[i]
end = omap[j - 1] + utf8_len_at(canon, omap[j - 1])   # end_exclusive
assert 0 <= start < end <= len(canon)
evidence = Span(revision_id, start, end, sha256(canon[start:end]))
```

Every Evidence is validated before publication by re-extracting the range bytes and matching the hash (`FT-11`).

10 Deletion, Retention and Immutability

DESIGN

Revision immutability appears to conflict with deletion requests and retention policy. The proposed resolution is “logical immutability, cryptographic deletion”. The content of each Revision is encrypted with a data key under envelope encryption.

The key granularity is an open design decision to be settled in `P3b`: a per-document key allows one document to be deleted without affecting others, while destroying a per-Tenant key would make all of that Tenant's content unrecoverable. Deletion means destroying the applicable key together with recording a content-free Tombstone. The hash and audit metadata remain. The content bytes become unrecoverable only if every copy of the key (including backups, replicas and escrow) is destroyed and no plaintext copy survives in caches, logs or projections, which are purged separately. This guarantee must be verified by test and legal review.

- Projections (graph, text, vector) and caches must be invalidated and rebuilt on a deletion event; dependent citations are marked invalid.
- Claims derived from deleted content change to “source unavailable” status and do not appear in answers.
- The retention policy is configurable per Tenant and Space; backups are managed with encryption keys separate from live content.
- Audit records the deletion event without keeping the deleted content.

- The feasibility of this scheme and its compliance with regulation (such as the right to erasure) has not yet been legally reviewed (`AUD-09`).

11 Evaluation and Execution Evidence

DESIGN EVIDENCE

Status: EVIDENCE: NOT RUN. No software test has been executed for this document.

Area	Metrics
Retrieval	Recall@K, nDCG@K, graph path validity, authorization isolation
Answer	Citation precision, Faithfulness, abstention accuracy, conflict handling, resistance to prompt injection
Operations	P95 latency, cost per answer, publication consistency, queue backlog, failure and retry rate, Restore success, observed RPO/RTO

Evaluation sets must include positive, negative, cross-tenant, revoked, conflicting and stale-Publication samples. Thresholds must be set on a representative workload and approved before release; invented percentages are forbidden.

Threshold-setting method

- Step 1: the owner defines the representative set (number of documents, language distribution, query patterns and expected concurrent load).
- Step 2: a baseline is measured on this set and recorded with all details.
- Step 3: the acceptance threshold is agreed with the owner and recorded, signed, in the Evidence Pack.
- Volume, QPS and latency targets remain undefined until Step 1 is done, and no number is guessed anywhere.

Evidence record

Test identifier, requirement identifier, code version, dataset and version, environment, commands, timestamps, expected and actual output, pass/fail, artifact hash and reviewer. A designed test is not execution evidence.

12 Disaster Recovery and Proposed Targets

ASSUMPTION ADR

A single-region deployment with backups in an independent failure domain and Point-in-Time Recovery is proposed. An RPO of at most 15 minutes, an RTO of at most 4 hours and an index rebuild within 24 hours are proposed targets only, not measured results. A Restore test and practical measurement are required before software release. The reference PostgreSQL version for recovery is 16; the final version must be pinned at implementation time and checked against the documentation of that version.

13 Dual-Mode Delivery Roadmap

ADR DESIGN

Phase	Content	Gate
P0	Engineering foundation: repository structure, module contracts, local environment, CI, base schema, observability skeleton	Reproducible build and architecture checks

Phase	Content	Gate
P1	Identity and Tenant isolation: OIDC, server-derived scope, policies, RLS defense, negative tests	All mandatory isolation tests on a real database and pool
P2	Knowledge and ontology: document registry, stable identifiers, schema validation, repository/space Bindings, controlled ontology	Deterministic schema and link behavior
P3a	Ingestion and provenance: immutable Revisions, idempotent asynchronous processing, extraction, evidence	Fault-injection and idempotency tests
P3b	Publication: projection consistency, Manifest and atomic activation, deletion and retention	Fault-injection and rollback tests
P4	Retrieval and GraphRAG: FTS, vector, bounded graph, Fusion, permission-aware cache, evaluation harness, Persian processing	Permission and Publication consistency and a retrieval baseline
P5	Answering and verification: three-route policy, citations, claim support, abstention, injection defense	Evidence and adversarial evaluation
P6	Production hardening: backup/PITR, Restore exercise, security review, runbooks, cost/latency monitoring	Measured recovery and risk acceptance
P7	Release qualification: Evidence pack, traceability, SBOM, versioned API, release and rollback exercise	Owner approval with no hidden exceptions

A development agent or a human engineering team must follow the same contracts, tests and approval gates. No mode can remove Evidence, a gate, or owner approval. This document alone does not grant permission to implement.

14 Formal 32-Requirement Matrix and Acceptance Criteria

REQ

The requirement text is the reconstructed, approved Baseline 1.0 and is not presented as a verbatim quotation of a historical version. Each requirement has exactly one designed test.

Functional requirements (FR)

ID	Requirement	Test / Phase	Acceptance criterion
REQ-FR-01	Register and manage Markdown knowledge documents and sources	FT-01 / P2	A valid document is registered and retrievable; an invalid document is rejected
REQ-FR-02	Maintain immutable versions of ingested sources	FT-02 / P3a	A source version never changes after it is recorded, and its hash remains valid

ID	Requirement	Test / Phase	Acceptance criterion
REQ-FR-03	Validate structure and frontmatter	FT-03 / P2	Frontmatter that does not conform to the schema is rejected
REQ-FR-04	Manage structured metadata and schema versions	FT-04 / P2	The schema version is recorded and incompatibility is detected
REQ-FR-05	Preserve stable identifiers independent of path and file name	FT-05 / P2	A rename does not change the document identifier; a duplicate identifier is rejected
REQ-FR-06	Support direct links between documents	FT-06 / P2	A valid link resolves; an invalid link has an explicit status
REQ-FR-07	Generate backlinks from primary relations	FT-07 / P2	Backlinks update correctly after a link is added or removed
REQ-FR-08	Support a controlled, extensible ontology	FT-08 / P2	An incompatible or unapproved extension is not activated
REQ-FR-09	Extract and separate semantic entities	FT-09 / P3a	An ambiguous entity merge does not happen without approval
REQ-FR-10	Extract relations with valid evidence	FT-10 / P3a	A relation without valid evidence is not published as a confirmed fact

ID	Requirement	Test / Phase	Acceptance criterion
REQ-FR-11	Record provenance and the exact location of evidence	FT-11 / P3a	Every piece of evidence links to a retrievable Revision and Span
REQ-FR-12	Version knowledge and semantic relations	FT-12 / P3a	A version change preserves history and relation status
REQ-FR-13	Publish a consistent knowledge set atomically	FT-13 / P3b	A partial indexing failure does not cause a partial publication
REQ-FR-14	Full-text and structured search	FT-14 / P4	Full-text search returns only authorized results
REQ-FR-15	Versioned vector search	FT-15 / P4	Embeddings incompatible with the index model or dimension are not mixed
REQ-FR-16	Graph traversal with limits and authorization	FT-16 / P4	Traversal never exceeds the depth, time and permission bounds
REQ-FR-17	Combine keyword, vector and graph retrieval	FT-17 / P4	Combined results belong to one valid Publication

ID	Requirement	Test / Phase	Acceptance criterion
REQ-FR-18	Generate evidence-based, policy-governed answers	FT-18 / P5	An answer is issued only through a route permitted by the routing policy
REQ-FR-19	Validate citations and claim support	FT-19 / P5	An invalid, unauthorized or unsupported citation is rejected
REQ-FR-20	Evaluate quality, cost, latency and abstention	FT-20 / P5–P7	Quality and cost metrics are reported against a reference test set

Multi-repository and multi-space requirements (MR)

ID	Requirement	Test / Phase	Acceptance criterion
REQ-MR-01	Support multiple repositories in one system	MT-01 / P2	Several independent repositories are registered and processed without interference
REQ-MR-02	Support multiple knowledge spaces	MT-02 / P1	Several Spaces are created with independent identity and settings
REQ-MR-03	Attach multiple repositories to one space	MT-03 / P2	Several repositories are bound to one Space and their permissions are honored

ID	Requirement	Test / Phase	Acceptance criterion
REQ-MR-04	Attach one repository to multiple spaces	MT-04 / P2	One repository is used in several Spaces without data leakage
REQ-MR-05	Apply independent authorization to each repository binding	MT-05 / P1–P2	An unauthorized Binding creates no access
REQ-MR-06	Isolate tenant data and identity	MT-06 / P1	Tenant A cannot see Tenant B's private data or metadata
REQ-MR-07	Isolate search indexes	MT-07 / P4	Full-text and vector search never leave the authorized scope
REQ-MR-08	Isolate access to graph relations	MT-08 / P4	Graph traversal does not reveal an unauthorized relation
REQ-MR-09	Prevent information leakage in answers and citations	MT-09 / P5	Answers, citations, result counts and cache reveal no unauthorized information
REQ-MR-10	Publish an independent version per space	MT-10 / P3b	Publishing one Space does not change another Space's active Publication
REQ-MR-11	Apply permission revocation immediately on read paths	MT-11 / P1–P5	After revocation, requests and caches no longer grant the former access

ID	Requirement	Test / Phase	Acceptance criterion
REQ-MR-12	Record an audit of accesses and changes	MT-12 / P1–P7	Sensitive events have an auditable Audit Record

Measurable conditions for test automation

The tables below turn each approved acceptance criterion into a binary, automatable condition and do not replace the approved text. In case of a difference, the approved acceptance criterion prevails.

ID	Test	Measurable condition (oracle)
REQ-FR-01	FT-01	Every valid sample in the golden set is registered and retrievable, and every invalid sample is rejected with a specific error code.
REQ-FR-02	FT-02	Any attempt to write to a recorded Revision is rejected, and the recomputed hash equals the stored hash.
REQ-FR-03	FT-03	Any frontmatter outside the schema is rejected with a recorded reason; conforming frontmatter is accepted.
REQ-FR-04	FT-04	Each document's schema version is recorded, and incompatibilities between versions are surfaced in an explicit report.
REQ-FR-05	FT-05	After a path rename the identifier is unchanged, and registering a duplicate identifier fails with an error.

ID	Test	Measurable condition (oracle)
REQ-FR-06	FT-06	A valid link resolves to the correct document, and an unresolved link produces an explicit diagnostic (not a silent failure).
REQ-FR-07	FT-07	After a link is added or removed, the set of backlinks exactly equals the currently authorized links.
REQ-FR-08	FT-08	An incompatible extension, or one without reviewer approval, is never activated; an approved extension is activated.
REQ-FR-09	FT-09	Every ambiguous merge stays in the review queue until approved; an approved merge is reversible.
REQ-FR-10	FT-10	No SUPPORTS or CONTRADICTS relation without valid Evidence appears in an approved Publication.
REQ-FR-11	FT-11	For each Evidence, the Revision is verified against its Revision hash, the span bytes are re-extracted within bounds and matched against the span hash when one is stored; an out-of-range span is rejected.
REQ-FR-12	FT-12	After a version change, the history and prior status of relations remain retrievable.
REQ-FR-13	FT-13	With a failure injected into any projection, the previously active Manifest stays unchanged and the incomplete version is not observable.
REQ-FR-14	FT-14	No result outside the user's authorized scope appears in search output.
REQ-FR-15	FT-15	Insertion or search with an incompatible model or dimension is rejected with an error.

ID	Test	Measurable condition (oracle)
REQ-FR-16	FT-16	Traversal is cut off at each of the three bounds (depth, time, permission) and an event is recorded.
REQ-FR-17	FT-17	All combined results share one Manifest identifier.
REQ-FR-18	FT-18	Every answer shows exactly one route (A, B or C), and any route outside policy is rejected.
REQ-FR-19	FT-19	Any citation with an invalid span or hash, a revoked authorization, or an unsupported claim is rejected.
REQ-FR-20	FT-20	A report of all metrics in Section 11 is produced against the reference set and the owner-approved thresholds.

ID	Test	Measurable condition (oracle)
REQ-MR-01	MT-01	Registering at least two repositories concurrently does not affect each other's identifiers or results.
REQ-MR-02	MT-02	Multiple Spaces are created with independent identifiers and configuration; changing one has no effect on another.
REQ-MR-03	MT-03	The permission of each repository is applied separately in the combined Space, and results are filtered accordingly.
REQ-MR-04	MT-04	No data from Space A appears in the answers of Space B, which has no permission.
REQ-MR-05	MT-05	A Binding without authorization reveals no result or metadata.

ID	Test	Measurable condition (oracle)
REQ-MR-06	MT-06	All cross-tenant attempts in the Section 5 negative tests are rejected, without exception, on a real database and connection pool.
REQ-MR-07	MT-07	No full-text or vector result outside the authorized scope is returned.
REQ-MR-08	MT-08	No unauthorized edge or node appears in any traversal path.
REQ-MR-09	MT-09	For a requester without permission, answers, citations, counts and cache expose no information.
REQ-MR-10	MT-10	After Space A is published, Space B's active Manifest pointer is unchanged.
REQ-MR-11	MT-11	A request after revocation is rejected even with an old Manifest and cache.
REQ-MR-12	MT-12	Every listed sensitive event has one queryable audit record that contains no confidential content.

15 Technical Scenarios and Negative Tests

DESIGN

Scenario	Test	Expected behavior
Document rename	FT-05 / FT-06	If a file path changes, the stable identifier and [[document-id]] links must remain valid; a duplicate identifier is rejected.

Scenario	Test	Expected behavior
Invalid evidence	`FT-11` / `FT-19`	If the Revision hash or a stored span hash does not match, or the evidence byte range is out of bounds, the citation is not published and the answer is corrected or rejected.
Failure midway through publication	`FT-13` / `MT-10`	If the vector index is built but the graph fails, the previous active Manifest is kept and the incomplete version is not observable.
Revocation at query time	`MT-09` / `MT-11`	After revocation, even an old cache or Manifest must not reveal a private citation or snippet.
Prompt injection in retrieved text	`FT-18` / `MT-06`	Source text that asks to disclose data or change policy is treated purely as data; the answer is issued only through the permitted route.
Injection against the claim judge	`FT-19` / `FT-18`	A claim or range containing instructions to the judge must not change its behavior: the instruction is not followed, no tool or permission is used, policy is unchanged and the output stays within the closed schema; with insufficient evidence the answer is rejected or abstains. An output outside the schema is rejected.
Cryptographic deletion	`FT-02` / `MT-11`	After a Revision's key is destroyed, its bytes cannot be recovered, and dependent projections, caches and citations are invalidated.

Scenario	Test	Expected behavior
Persian normalization	FT-11 / FT-14	A search for می‌شود (with ZWNJ) and می شود (with a plain space) and for Persian versus ASCII digits gives one identical result, while the evidence offset still points at the canonical bytes.

16 Audit Controls

AUDIT

Control	Contract status	Proof status
AUD-01	The reconstructed 32 requirements were approved as Baseline 1.0; no claim of verbatim identity with the historical text is made.	Closed at contract level, with owner approval
AUD-02	Tenant/Space isolation contract defined.	Executable proof: pending
AUD-03	RLS, pooling and revocation test contract defined.	Execution: pending
AUD-04	Immutable-Revision and UTF-8 span contract defined.	Implementation: pending
AUD-05	Publication-consistent recovery and an independent backup domain specified.	Exercise: pending
AUD-06	Retrieval and answer evaluation defined.	Measured baseline: pending

Control	Contract status	Proof status
AUD-07	Versioned OpenAPI and compatibility policy specified.	Implementation: pending
AUD-08	Recovery targets are assumptions.	Awaiting measurement
AUD-09	Rights-notice policy for this PDF approved.	Enforceability, signature and legal analysis: pending
AUD-10	Official external references listed.	Version verification and claim-to-citation mapping at final release: pending

> Final verdict: internal design-review status “PASS” at design and documentation level only; no independent verification. Production readiness: not approved. Legal enforceability: pending review. Tests executed: none.

17 Technical References and Citation Status

REF

The seven addresses below are candidate references. Matching them against the publishers' official pages, pinning versions and mapping each claim to its source are performed at final release (AUD-10). These references are not proof of implementation or

compliance. `REF-02` is deliberately pinned to version 16 (the recovery target); the others point to the current page and must be pinned to an approved version at implementation time.

ID	Source	Use	Address
`REF-01`	PostgreSQL row-level security	RLS and data isolation	https://www.postgresql.org/docs/current/ddl-rowsecurity.html
`REF-02`	PostgreSQL continuous archiving / PITR (version 16)	WAL and PITR	https://www.postgresql.org/docs/16/continuous-archiving.html
`REF-03`	OpenAPI specification	API contract	https://spec.openapis.org/oas/
`REF-04`	OpenID Connect Core 1.0	OIDC identity	https://openid.net/specs/openid-connect-core-1_0.html
`REF-05`	OWASP LLM Prompt Injection Prevention Cheat Sheet	Prompt-injection defense	https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
`REF-06`	NIST SP 800-34 Rev.1	Contingency and recovery planning	https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final
`REF-07`	OpenTelemetry specifications	Tracing and observability	https://opentelemetry.io/docs/specs/otel/overview/

18 Rights Notice

AUDIT

Clause	Text
Author and rights holder	Amin Azimi — Azimi Innovation Lab
Publication	Published as a portfolio document for viewing and reference.
Permitted use	Reading and personal reference, with attribution to the author.
Restrictions	Republication, redistribution, transfer, sale, sublicensing, and reproduction for distribution are not permitted without prior written permission.
Use beyond this permission	Implementing the described idea or architecture, organizational use, and commercial exploitation require a separate written license from the rights holder.
Ownership	Rights in the text, diagrams and original expression of this document are reserved to the rights holder. This notice does not by itself create exclusive ownership of abstract ideas.
Legal status	This notice is a limited permission for use of the document and is not a substitute for a signed agreement. A license for any future software is determined separately.

19 Related Designs by the Same Author

REF

The two projects below, by the same author (Azimi Innovation Lab), apply patterns aligned with this architecture in another domain. Both have “Candidate” status; this section is an introduction and a statement of design alignment only, and it grants no right to use their code.

Project	Link	Alignment with this document
AlinaMatrix-Enterprise	https://github.com/aminbita162-glitch/AlinaMatrix-Enterprise	Immutable sources and fragments; quote-locked “claim atoms” with provenance (Sections 4 and 7); evidence and conflict checks before release (Section 7); Tenant isolation with PostgreSQL RLS (Sections 5 and 8); append-only audit and revocation events; a release gate with human approval.
AminOpsTwin	https://github.com/aminbita162-glitch/AminOpsTwin	A Policy Gateway with deny-by-default over every action; no direct agent access to the database; a durable executor for the effect of allowed actions (Sections 2 and 8); recorded denial reasons; phased development (compare Section 13).